

Лабораторна робота №8.

Основи JavaScript: типи даних і приведення типів, Scope, Closures, функції, масиви, об'єкти та ES Modules

Термін здачі: 09.10.2026

1. Мета роботи

Розібратися, де JavaScript поводиться інакше, ніж мови, які ви вже знаєте: приведення типів, області видимості й hoisting, замикання, значення й посилання, ES-модулі.

Навчитися перевіряти власну інтуїцію і впевнену відповідь AI-агента запуском коду, а не довірою.

Очікуваний результат: п'ять власноруч написаних чистих функцій, які проходять надані тести, і для кожної — пояснення, яку пастку мови вона обходить і чому.

2. Цілі та результати навчання

Після виконання лабораторної роботи студент повинен:

- Пояснювати різницю між примітивами й об'єктами, результати typeof і те, чому `0.1 + 0.2 !== 0.3`.
- Прогнозувати неявне приведення типів у `+`, `-`, `==`, `>=` і в умовах; обирати між `==` і `===`, `||` і `??`; застосовувати `??`.
- Пояснювати різницю між `var`, `let` і `const`, блочною та функціональною областю видимості, hoisting і temporal dead zone (TDZ).
- Пояснювати, що «запам'ятовує» замикання, і використовувати його для прихованого стану.
- Розрізняти копіювання значення й посилання, поверхневу й глибоку копію, методи масиву, що змінюють масив, і ті, що повертають новий.
- Використовувати параметри за замовчуванням, `rest` і деструктуризацію; пояснювати, чому значення за замовчуванням не спрацьовує для `null`.
- Пояснювати, чим ES-модуль відрізняється від класичного скрипта.

- Перевіряти чисті функції тестами `node --test` і читати повідомлення `assert`.
- Використовувати AI-агента як співрозмовника, відповідь якого перевіряють запуском, а не як автора коду.

3. Завдання лабораторної роботи

1. Опрацювати дослідницькі завдання (§4) і провести експерименти: для кожного записати власний прогноз і прогноз AI-агента **до** запуску.
2. Завантажити [starter-проект JS Basics](#), зафіксувати його в Git без змін і переконатися, що тести падають (§5.1).
3. Реалізувати функції Challenges 1–5 так, щоб пройшли всі тести (§5.3–5.7), — по одному коміту на challenge.
4. Опублікувати проєкт на GitHub Pages, оформити звіт (§6) і захистити роботу (§7).

Важливо! У цій роботі немає DOM, подій, таймерів і Promise — це теми ЛР-9 і Лекції 6. Функції в `src/` — чисті: отримують дані через параметри, повертають результат і нічого не змінюють поза собою.

4. Дослідницькі завдання

Перед практичною частиною опрацюйте джерела й проведіть експерименти. Загальна довідка — [JavaScript Guide — MDN](#); для глибшого читання — [You Don't Know JS Yet і ECMAScript 2025 Language Specification](#).

Протокол експерименту:

1. Запишіть **власний прогноз** результату.
2. Попросіть AI-агента дати прогноз, **не показуючи** йому результату й не дозволяючи запускати код.
3. Виконайте код у Console браузера (DevTools). Кожен фрагмент вставляйте цілим.
4. Якщо хоч один прогноз не збігся з результатом — поясніть, що саме неправильно передбачено.

Результати зведіть у таблицю — вона входить у звіт:

#	Код	Мій прогноз	Прогноз AI (без запуску)	Фактичний результат	Пояснення
1	'5' + 1	...	...	...	...

Біля кожного експерименту вказано дефект, до якого ця поведінка призводить у реальному коді. У поясненні напишіть, як цього дефекту уникнути.

4.1. Типи даних і приведення типів

Опрацювати:

- [JavaScript data types and data structures — MDN](#)
- [Equality comparisons and sameness — MDN](#)
- [Type coercion](#) і [Falsy](#) — MDN Glossary
- [Nullish coalescing operator \(??\)](#) і [Optional chaining \(?.\)](#) — MDN
- [IsLooselyEqual — ECMAScript 2025](#) — алгоритм == у специфікації

Дослідити: які типи має JavaScript і що повертає typeof для кожного, зокрема для null, масиву й функції; які значення falsy; коли + склеює рядки, а коли додає числа; за якими правилами працює == і чому типовий вибір — ===; чим ?? відрізняється від ||; що повертає ?. , якщо ланцюжок обривається; чому NaN не дорівнює самому собі.

Експерименти:

1. '5' + 1 і '5' - 1

Дефект: значення поля форми — завжди рядок, тому «2» + 1 дає «21».

2. 0.1 + 0.2 === 0.3

Дефект: суми й середні значення не сходяться на останньому знаку.

3. [NaN === NaN, Number.isNaN(NaN), isNaN('abc'), Number.isNaN('abc')]

Дефект: перевірка x === NaN ніколи не спрацьовує.

4. [0 || 'немає', 0 ?? 'немає']

Дефект: оцінка чи ціна 0 показується як «немає даних».

5. [null == undefined, null == 0, null >= 0]

Дефект: null в одному порівнянні поводить як 0, а в іншому — ні.

6. [10, 9, 1].sort()

Дефект: масив чисел сортується як рядки.

4.2. Scope, Hoisting і Closures

Опрацювати:

- [Grammar and types — MDN](#), розділ Declarations
- [let — MDN](#), розділ Temporal dead zone
- [Hoisting — MDN Glossary](#)
- [Closures — MDN](#)
- [You Don't Know JS Yet: Scope & Closures](#)

Дослідити: чим блочна область видимості відрізняється від функціональної; що саме «піднімається» під час hoisting для var, let/const і function declaration; що таке TDZ; чому const не робить об'єкт незмінним; що таке замикання і що воно зберігає —

значення змінної в момент створення чи саму змінну; скільки окремих змінних створює `for (let i ...)`, а скільки — `for (var i ...)`.

Експерименти:

7. `console.log(early); var early = 1;`, а потім окремо `{ console.log(late); let late = 1; }` — у блоці, бо `let` верхнього рівня Console обробляє по-своєму.

Дефект: з `var` код мовчки працює з `undefined` замість того, щоб упасти там, де помилка.

8. Цикл, що створює функції:

```
const viaVar = [];  
for (var i = 0; i < 3; i++) viaVar.push(() => i);  
const viaLet = [];  
for (let j = 0; j < 3; j++) viaLet.push(() => j);  
[viaVar.map((f) => f()), viaLet.map((f) => f())];
```

Дефект: усі обробники, створені в циклі, бачать останнє значення лічильника.

9. `const config = { theme: 'light' }; config.theme = 'dark'; config;`, а потім `config = {};`

Дефект: `const` сприймають як незмінність, а об'єкт змінюється з будь-якого місця, де є посилання на нього.

4.3. Значення й посилання: об'єкти та масиви

Опрацювати:

- [Working with objects — MDN](#)
- [Shallow copy](#) і [Deep copy](#) — MDN Glossary
- [structuredClone\(\) — MDN](#)
- [Object.freeze\(\) — MDN](#)
- [Array: copying methods and mutating methods — MDN](#)
- [Map — MDN](#)

Дослідити: що копіює присвоєння `b = a` для примітиву й для об'єкта; що порівнює `===` для двох об'єктів; що копіює `spread` (`{ ...obj }`, `[...arr]`) і чого не копіює; коли потрібен `structuredClone`; що робить `Object.freeze` і що стається під час спроби змінити заморожений об'єкт у `strict mode` і без нього; які методи масиву змінюють сам масив (`sort`, `reverse`, `splice`, `push`), а які повертають новий (`toSorted`, `toReversed`, `map`, `filter`, `slice`); чим `Map` відрізняється від звичайного об'єкта як словник і що може бути його ключем.

Експерименти:

10. Поверхнева копія:

```
const user = { name: 'Ann', address: { city: 'Kyiv' } };
const copy = { ...user };
copy.name = 'Bob';
copy.address.city = 'Lviv';
[user.name, user.address.city];
```

Дефект: «копію» редагують у формі, а змінюється оригінал, бо вкладений об'єкт спільний.

11. Заморожування:

```
const defaults = Object.freeze({ theme: 'light', limits: { perPage: 20 }
});
defaults.theme = 'dark';
defaults.limits.perPage = 100;
defaults;
```

Потім повторіть фрагмент з іменем `strictDefaults` замість `defaults` і рядком `'use strict'`; на початку.

Дефект: «заморожені» налаштування все одно змінюються через вкладений об'єкт. Модулі завжди виконуються в `strict mode` — так тести `starter-проєкту` ловлять функцію, яка змінює свої аргументи.

12. `const ratings = [8.2, 9.1, 7.5]; const sorted = ratings.sort(); [sorted === ratings, ratings];`, а потім `const scores = [8.2, 9.1, 7.5]; const ranked = scores.toSorted(); [ranked === scores, scores];`

Дефект: масив відсортували «для показу», а змінився вихідний, з яким працює інший код.

4.4. Функції та ES Modules

Опрацювати:

- [Functions — MDN](#)
- [Arrow function expressions — MDN](#)
- [Default parameters, Rest parameters, Destructuring](#) — MDN
- [JavaScript modules — MDN](#), зокрема [Other differences between modules and classic scripts](#)
- [Strict mode — MDN](#)

Дослідити: чим відрізняються `function declaration`, `function expression` і `arrow function`; коли спрацьовує значення параметра за замовчуванням; як працюють `rest-параметри` й

деструктуризація об'єкта зі значеннями за замовчуванням; чим `named export` відрізняється від `default`; чим ES-модуль відрізняється від класичного `<script>: strict mode`, власна область видимості верхнього рівня, `this` на верхньому рівні, завантаження лише через HTTP(S).

Експерименти:

13. `function greet(name = 'гість') { return name; } [greet(), greet(undefined), greet(null)];`

Дефект: API повертає `null` для відсутнього поля, і значення за замовчуванням не спрацьовує.

14. Виклик до оголошення:

```
{
  console.log(declared());
  console.log(arrow());
  function declared() { return 'declaration'; }
  const arrow = () => 'arrow';
}
```

Дефект: функцію перенесли в кінець модуля, переписавши як `arrow function`, — і код, що викликав її раніше, падає.

15. На сторінці `starter`-проєкту, відкритій через `Live Preview` (§5.1), тимчасово допишіть у кінець `main.js` рядки `var fromModule = 'module';` і `console.log('this:', this);`, оновіть сторінку й виконайте в `Console` `var fromConsole = 'console';` `[window.fromModule, window.fromConsole];`. Потім видаліть ці рядки з `main.js`.

Дефект: змінні верхнього рівня класичних скриптів потрапляють у `window`, і один скрипт перезаписує змінну іншого.

5. Практична частина: Code Challenges

Starter-проєкт **JS Basics** — п'ять модулів у `src/` із заглушками функцій, тести до них у `test/` і 30 серіалів із [Tvmaze](#) у `data/shows.json`. Що має робити кожна функція, описано в [ТЗ](#); тести перевіряють саме ці правила, і назва тесту — правило, яке він перевіряє. Виконуйте `challenges` по черзі: C2–C4 працюють з результатом C1.

5.1. Підготовка

1. Завантажте [js-basics-starter.zip](#) і розпакуйте в **окремий репозиторій** або в теку `lab-8/` спільного репозиторію з лабораторними. Зробіть перший коміт **без змін** — від нього рахуються всі `diff` на захисті.

2. У корені проєкту виконайте `node --test`. Усі тести мають упасти з `Error: Not implemented`. Встановлювати нічого не потрібно: `node:test` і `node:assert` входять у `Node.js`.
3. Відкрийте `index.html` подвійним кліком — адреса почнеться з `file://`. Звіту на сторінці не буде, а в Console з'явиться помилка `Access to script at 'file:///.../main.js' from origin 'null' has been blocked by CORS policy`. Поясніть у звіті, чому браузер не завантажує модуль, який лежить у тій самій теці.
4. Відкрийте ту саму сторінку через Live Preview (або `npx serve`). Тепер звіт є, але замість результатів у розділах — помилки: функції ще не реалізовано. Оновлюйте сторінку після кожного challenge.

5.2. Правила роботи з AI-агентом

- **Дослідження (§4).** AI дає прогноз **до** запуску коду. У таблицю записуйте його відповідь так, як він її дав, без виправлень після запуску.
- **Challenges.** AI може пояснити поняття, повідомлення тесту чи помилку, але **не пише** функцію — ні цілком, ні «підказкою» з готовим кодом. Якщо агент працює в IDE, використовуйте режим без редагування (`ask / plan`).

Приклад запиту:

Поясни, чому падає цей тест. Нижче — його назва, повідомлення `assert` і моя функція. Не пиши виправлений код: назви правило JavaScript, яке я, ймовірно, не врахував, і запропонуй, як перевірити це в Console.

На захисті ви зміните функцію наживо, без AI (§7). У звіті вкажіть назву та версію агента, модель і дату роботи.

5.3. Challenge 1 — `normalizeShow`

Перетворити запис `TVmaze` на плоский об'єкт, з яким працюють решта функцій. Дані неповні: у записах трапляються `null` і відсутні поля. Теми: §4.1, §4.3. Тести: `test/normalize.test.js`.

5.4. Challenge 2 — `filterShows`

Пошук за частиною назви, жанром і мінімальною оцінкою; кожен фільтр може бути не заданий. Теми: §4.1, §4.4. Тести: `test/filter.test.js`.

5.5. Challenge 3 — sortShows

Сортування за назвою, роком або оцінкою в обох напрямках, без зміни вхідного масиву. Теми: §4.1, §4.3. Тести: `test/sort.test.js`.

5.6. Challenge 4 — genreStats

Кількість серіалів і середня оцінка для кожного жанру. Теми: §4.1. Тести: `test/stats.test.js`.

5.7. Challenge 5 — createCounter, once, memoize

Три функції, що зберігають стан у замиканні. Теми: §4.2. Тести: `test/closures.test.js`.

5.8. Коміти

Кожен challenge — **окремий коміт**, щойно його тести пройшли, наприклад `feat(C3): implement sortShows`. Файли в `test/` не змінюються в жодному коміті; тимчасові рядки з експерименту 15 у коміті не потрапляють. Після C5 опублікуйте проєкт на GitHub Pages — інструкція в `README.md` starter-проєкту.

6. Вимоги до звіту

Тема листа:

[Веб] ЛР-8: Основи JavaScript: типи даних і приведення типів, Scope, Closures, функції, масиви, об'єкти та ES Modules

У листі вкажіть:

GitHub repository: `https://github.com/<username>/<repository>`
GitHub Pages: `https://<username>.github.io/<repository>/`

Звіт повинен містити:

1. таблицю експериментів §4 (усі 15) і короткі висновки до кожного підрозділу: де підвела інтуїція з інших мов, а де помилився AI;
2. назву та версію AI-агента, модель, дату роботи;
3. пояснення з §5.1: чому `index.html` не працює через `file://` і працює через Live Preview;
4. вивід `node --test` після останнього коміту — усі тести пройшли;
5. для кожного challenge: пастку, яку він містив (який тест упав на вашій першій реалізації, а якщо жоден — від чого захищає ваш код), правило JavaScript, що за нею

стоїть, і чому ваше рішення працює;

6. підсумок: у яких експериментах помилився AI і чому, на вашу думку.

7. Усний захист

Формат задачі передбачає усну співбесіду за результатами практичної реалізації. Функції мають бути написані вами: AI міг пояснювати, але не писати код. Тому на захисті ви змінюєте код наживо, без AI, і відповідаєте на питання з демонстрацією екрана.

Орієнтовні питання:

Challenges

- Чому тест X падав до вашого виправлення? Покажіть на відповідному коміті.
- Змініть функцію наживо: наприклад, `null` на початку замість кінця або сортування за назвою за спаданням. Які тести це зачепить і чому?
- Чому тест падає з `TypeError: Cannot assign to read only property?`

Типи та приведення

- Чим `??` відрізняється від `||`, а `==` від `===`? Чому `null >= 0` — `true`, а `null == 0` — `false`?
- Чому `0.1 + 0.2 !== 0.3` і як це враховує ваш C4?

Scope і closures

- Спрогнозуйте без запуску цикл із замиканнями з `var`, потім з `let`. Поясніть різницю.
- Що «пам'ятає» `createCounter` і чому два лічильники не заважають один одному?

Посилання та модулі

- Що копіює `spread`, а що — ні? Коли потрібен `structuredClone`?
- Чому значення параметра за замовчуванням не спрацьовує для `null`?
- Чому `index.html` не працює з `file://`, а через Live Preview працює? Чим модуль відрізняється від класичного скрипта?

AI

- У якому експерименті помилився AI? Чому, на вашу думку?

8. Оцінювання

1. Успішний захист — **2 бали**.

2. Додатковий **1 бал** за однієї з умов:

- термін здачі до **09.10.2026**, АБО

- робота на парі.